gRPC + Protocol Buffers

What is gRPC?

gRPC is a **high-performance, cross-language, open-source framework** for **remote procedure calls (RPC)**. Created by Google, built on:

- Faster and more compact than REST/JSON
- Strongly typed interfaces
- Automatic client/server code generation
- First-class interop (Java, Go, Python, C++, C#, Kotlin...)

When to use

- Microservice-to-microservice communication (low-latency)
- High-throughput services
- IoT / robotics / Edge computing
- Strongly typed enterprise systems

Not ideal for

- Public APIs (JSON is easier)
- Browsers (no direct gRPC support)
- Simple CRUD applications

Protocol Buffers (Protobuf)

A **binary serialization format** with:

- Very compact data representation
- Code generation for strongly typed languages
- Strict backward/forward compatibility rules
- Schema defined in `.proto` files

Why Protobuf is Fast

Compact Binary Encoding

- Field names **not included** → only numeric tags
- Variable-length integer encoding
- Very small payloads (5–10× smaller than JSON)

Efficient Parsing

- Messages mapped to classes (precompiled code)
- Zero-copy deserialization patterns available

---

gRPC Service Definition

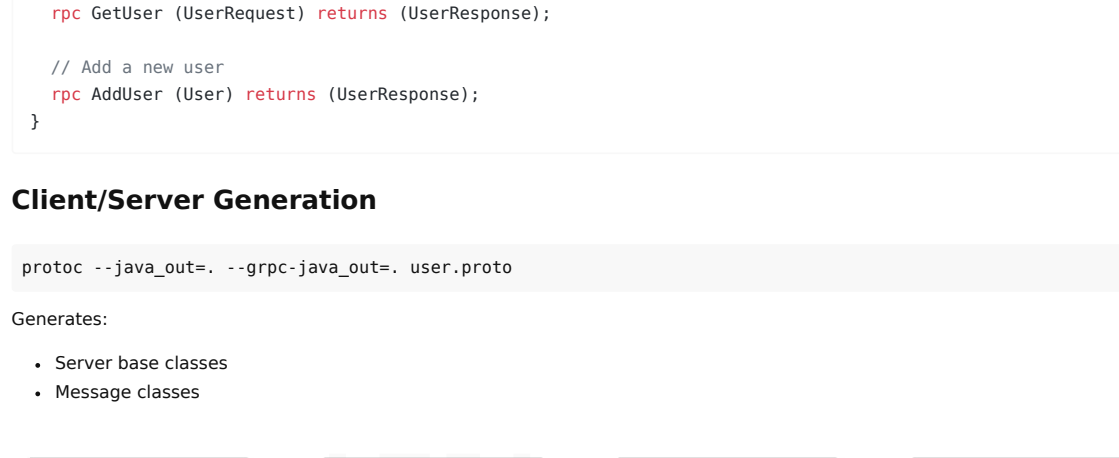
```
syntax = "proto3";

package user;

// ----------------------
// Messages (Entities)
// ----------------------

message Post {
  string timestamp = 1;
  string content   = 2;
}

message User {
  string userUUID  = 1;
  string nickname  = 2;
  string birthDate = 3;
  repeated Post posts = 4; // a user can have multiple posts
}

// Request and Response messages
message UserRequest {
  string userUUID = 1;
}

message UserResponse {
  User user = 1;
}

// ----------------------
// Service Definition
// ----------------------

service UserService {
  // Get a user by UUID
  rpc GetUser (UserRequest) returns (UserResponse);

  // Add a new user
  rpc AddUser (User) returns (UserResponse);
}
```

Client/Server Generation

```
protoc --java_out=. --grpc-java_out=. user.proto
```

Generates:

- Server base classes
- Message classes


Create .proto file to define data structure
.proto file
Input
Generate code using the protoc compiler
Output
.java, .py, .cc, or other source files
Input
Compile PB code with your project code
Output
Compiled classes
Input
Use PB classes to serialize, share, & deserialize data

Schema Evolution in Protobuf

Compatible Changes

- Adding optional fields
- Renaming a field (if tag number stays the same)
- Removing a field (keep the tag reserved)
- Changing default values (clients unaffected)

Breaking Changes

- Changing a field's type
- Reusing tag numbers
- Changing field labels (`repeated` → non-repeated)
- Removing fields without reserving tags

---

gRPC in Java (Spring Boot)

Server Implementation

```
@GrpcService
public class UserServiceImpl extends UserServiceGrpc.UserServiceImplBase {
    @Override
    public void getUser(UserRequest req, StreamObserver<UserResponse> res) {
        UserResponse response = ...
        res.onNext(response);
        res.onCompleted();
    }
}
```

Client Usage

```
ManagedChannel channel = ManagedChannelBuilder
    .forAddress("localhost", 50051)
    .usePlaintext()
    .build();

UserServiceGrpc.UserServiceBlockingStub client =
    UserServiceGrpc.newBlockingStub(channel);

UserResponse res = client.getUser(
    UserRequest.newBuilder().setId("123").build()
);
```

Resources